

A FIELD MANUAL FOR SMART CONTRACT AUDITORS



OFFICER'S BIG AUDITING BOOK

Reentrancy, oracles, DeFi integrations, fuzzing, and 30+ more
field notes for Solidity smart-contract auditors

officercia.eth

Compiled from the Officer's Notes / Pessimistic Security blog series

Officer's Big Auditing Book

A compiled field manual on smart-contract auditing — reentrancy, oracles, DeFi integrations, fuzzing, gas optimization, and more.

I want to start by thanking the pessimistic.io auditors, whose work and shared knowledge shaped so much of the experience behind these chapters — a great deal of what follows exists because of what I learned working alongside their team. I also want to thank the hexens.io team for their own contributions to this field and to the wider security community that these notes draw on.

This book gathers together a series of articles originally published as part of an ongoing series on smart-contract security — covering everything from classic reentrancy and oracle manipulation to protocol-specific integration notes for Aave, Compound, Curve, Convex, and Balancer, plus tooling chapters on Slither, Echidna, Gas Gauge, and Glider. Each chapter here is a rewritten summary of the original piece; the full original article, with its complete code samples and full resource lists, is linked at the end of every chapter for anyone who wants to go deeper.

Support this work

Writing and maintaining this material takes real time, and donations directly fund the ability to keep doing it. If this book was useful to you, please consider sending a donation to:

0x1191b7d163bde5f51d4d2c1ac969d514fb4f4c62

officercia.eth (ETH and EVM-compatible networks)

Thank you for reading, and stay safe.

Contents

01. Reentrancy Attacks on Smart Contracts Distilled
02. Auditing Projects on the NEAR Blockchain: From Zero to Hero
03. Oracles, Entropy & Chainlink VRF Secure Integration Tips
04. Chainlink VRF Secure Integration Tips: Specifications
05. Price & Reward Manipulation Attacks Distilled
06. Fuzzing Solidity Smart Contracts with Echidna
07. Slither: An Auditor's Cornucopia
08. Auditor's Notes: Semantic Grep & Solidity
09. Semantic Grep & Solidity: In-Depth
10. Read-only Reentrancy: In-Depth
11. AMM (Automatic Market Makers) Integration Tips
12. Short Types in Solidity: Rare Tricks Uncovered
13. Gas Gauge: Pressure Control
14. Arbitrary Calls Auditing Tips
15. EVM Limitations & Assembly Auditing Tips
16. Math, Solidity & Gas Optimizations
17. ERC20 Integration Tips
18. Solidity Checklist & Reentrancy Attack
19. Initializing, Proxy, Oracles & Multi-Chain
20. Tokens, EIP-712 & Meta-Transactions
21. AAVE V3 DeFi Integration Tips
22. AAVE V3 DeFi Integration: Specifications
23. Compound v2 DeFi Integration
24. Compound v2 DeFi Integration: Specifications
25. Convex Finance DeFi Integration Tips
26. CurveV1 Integration Tips
27. Auditing Tips for NFT Projects
28. BalancerV1 Integration Tips
29. Arbitrum: Basic Features, Technical Details and Differences from Ethereum
30. Rebase Tokens, Rounding Errors & DoS Attacks: Unraveling the Mystery
31. Leap Seconds & Time in Solidity
32. Ethereum Alarm Clock Exploit: Final Thoughts
33. 25 Tips for Solidity Code Auditors
34. Rust & Blockchain Security
35. Variant Analysis & Glider Tool
36. Optimizing the Auditing Process in Your Auditing Company
37. How Does a Compiler Actually Work?

CHAPTER 01

Reentrancy Attacks on Smart Contracts Distilled

Reentrancy is one of the oldest and still most dangerous bug classes in smart-contract security. The chapter opens with the core mechanic: an external call handed to an attacker-controlled address lets that address call back into the original contract before its state has finished updating, most famously when a withdrawal function sends funds before zeroing out the caller's balance. It walks through the three usual flavors auditors run into — single-function, cross-function, and cross-contract reentrancy — and shows a minimal vulnerable *DepositFunds* contract alongside the matching attacker contract that drains it through its fallback function. On the defensive side it favors the checks-effects-interactions ordering (update state before making external calls) and a simple boolean reentrancy-guard modifier, backed up by static tools such as Slither, Sereum, and Mythril for catching the pattern before deployment. It closes by flagging read-only reentrancy — where a view function returns stale data mid-reentry — as a newer, easy-to-miss variant worth its own dedicated look.

Read the original article: https://x.com/officer_secret/status/1932857048288670084?s=20

CHAPTER 02

Auditing Projects on the NEAR Blockchain: From Zero to Hero

NEAR is a sharded, proof-of-stake layer-one where contracts compile to WebAssembly and can be written in Rust, AssemblyScript, Solidity (via the Aurora EVM), or JavaScript — in practice, the ecosystem leans heavily Rust. The chapter lays out the audit procedure used on NEAR engagements: an automated pass (compiling the project, running the test suite and checking coverage, then running analysis tools and manually triaging every flagged issue) followed by a manual review that checks code quality, known vulnerability classes, and whether the implementation actually matches its documentation. It lists the issue categories auditors watch for — broken access control, stuck or misdirected assets, logical or gas-based denial of service, unsafe cross-contract calls, and arithmetic errors — plus NEAR-specific gas traps such as choosing the wrong collection type (LookupMap vs UnorderedMap vs TreeMap) or forgetting to sync a collection to storage. Rust tooling gets a dedicated section: Cargo Geiger for spotting unsafe code, Cargo Audit for vulnerable dependencies, Clippy as a linter, and Cargo Tarpaulin for coverage.

Read the original article: https://x.com/officer_secret/status/1933609227450249371?s=20

CHAPTER 03

Oracles, Entropy & Chainlink VRF Secure Integration Tips

This piece is a systematization of knowledge on oracles and randomness aimed at both developers and auditors. It defines a blockchain oracle as the bridge that gathers, verifies, and delivers off-chain data on-chain, with Chainlink as the running example of a decentralized oracle network. From there it pivots to entropy: the difference between low-entropy (predictable) and high-entropy (genuinely random) data-generating processes, and why on-chain 'randomness' derived from block hashes or gas cost is fundamentally gameable — an attacker can simply wait for a favorable value or, in extreme cases, buy the outcome they want. Three legitimate paths to real entropy are laid out: pulling from an external high-entropy API (even a consumer-grade RNG like atmospheric noise or a networked pet-based sensor rig makes the point), using Chainlink VRF as the standard production-grade solution, or collecting contributed randomness the way ceremony-style protocols do. The chapter then walks through Chainlink VRF's request-and-receive cycle, why its cryptographic proof matters, and a set of integration tips covering safe confirmation windows, never re-requesting randomness, and never accepting new bets after a randomness request has been made.

Read the original article: https://x.com/officer_secret/status/1934235993135059321?s=20

CHAPTER 04

Chainlink VRF Secure Integration Tips: Specifications

A tighter, checklist-style companion to the earlier VRF chapter. It restates the concrete integration rules: match every randomness request to its fulfillment via *requestId*, choose a safe block-confirmation depth so a miner can't quietly discard an unfavorable result, never build in logic that lets a request be re-rolled, and make sure *fulfillRandomWords* can never revert (since the VRF service will not retry a failed callback). It also stresses inheriting from the correct VRF consumer base contract and closing off any window where users could place new bids after randomness has already been requested but not yet delivered. Two real exploit patterns illustrate the stakes: a state-dependent randomness bug where the final outcome depended on a mutable variable a user could wait out, and a re-rolling bug where a malicious NFT-receiver contract could revert an unfavorable mint and try again. A short comparison of VRF V1 versus V2 rounds out the chapter, covering the new subscription funding mode, higher gas limit, and configurable confirmation depth in V2.

Read the original article: https://x.com/officer_secret/status/1969316151462105515?s=20

CHAPTER 05

Price & Reward Manipulation Attacks Distilled

A two-part deep dive that first separates market manipulation (genuinely moving an asset's supply-and-demand price) from oracle exploits (an oracle simply reporting bad data), since the two get conflated constantly even though the fix for each is different. It runs through classic manipulation techniques — spoofing, ramping, bear raids, cross-market manipulation, wash trading, and front-running — and explains how flash loans remove the capital constraint that used to make large manipulations expensive, letting an attacker borrow, distort a price or exploit a bug, and repay everything within one transaction block. The chapter then works through nearly a dozen real incidents with a consistent problem/solution format, including Inverse Finance's Curve-balance-derived collateral price, a WDOGE-BNB pair drained via *skim()* on a deflationary token, and Moonwell's stale bridged-token price feed after the Nomad bridge hack. A parallel section on reward manipulation covers airdrop and staking-reward bugs — snapshot-less balance checks, double-counted claims, and unprotected admin functions that let an attacker seed and drain a reward pool in a single flash-loaned transaction. It closes with defensive layers: circuit breakers, contract update delays, manual kill switches, and active monitoring.

Read the original article: https://x.com/officer_secret/status/1934571606870749549?s=20

CHAPTER 06

Fuzzing Solidity Smart Contracts with Echidna

An introduction to property-based fuzzing for Solidity using Echidna, a Haskell tool that generates grammar-aware inputs from a contract's ABI and tries to break user-defined invariants rather than just crash the program. It frames the invariant types Echidna is good at catching — broken access control, reachable-but-forbidden state transitions, and arithmetic that lets a user manufacture free tokens — and notes that it works across Hardhat, Foundry, Truffle, and Vyper projects via crytic-compile. Echidna is described as Slither-powered under the hood, using static analysis up front to make its fuzzing campaigns more targeted, and it supports both a Truffle/Ethno-driven initialization flow and direct ABI-based calls for testing already-complex contract states. A recommended procedure is given for setting it up cleanly: compile first, run Slither to catch configuration issues early, and only then launch the fuzzing campaign, since doing it in that order surfaces most setup problems before they waste fuzzing time.

Read the original article: https://x.com/officer_secret/status/1937492411300643089?s=20

CHAPTER 07

Slither: An Auditor's Cornucopia

Opens a series on audit methodology by making the case for Slither, Trail of Bits' static analyzer, as a baseline tool for maintaining audit quality. Beyond its vulnerability detectors, the chapter highlights Slither's printers, which summarize a contract's structure, inheritance, storage layout, modifiers, and implemented checks — useful for building a mental model of an unfamiliar codebase before diving into the logic line by line, though the printers get noisy fast on genuinely large projects. It also flags Slither's Python API, which lets an auditor script custom static-analysis passes on top of the engine rather than being limited to the built-in detector set, and mentions Slither-simil as a machine-learning-assisted way to extend detection further. The chapter frames Slither alongside SmartCheck (an older Pessimistic-built tool) and Echidna-based fuzzing as three complementary layers rather than competing options, with the underlying point that a new tool isn't automatically better than an old one — what matters is knowing when to reach for each.

Read the original article: https://x.com/officer_secret/status/1937869848056394106?s=20

CHAPTER 08

Auditor's Notes: Semantic Grep & Solidity

Kicks off a mini-series on pattern-matching tools that sit between plain-text grep and full static analysis. Solgrep is introduced as a Solidity-native option: it recursively parses a target directory's contracts, understands basic language semantics, and exposes that structure to a JavaScript-based filter function, which makes it easy to answer questions like 'which contracts expose a public *withdrawEth* method' without drowning in lexical false positives. Its downsides are just as clearly stated — it isn't especially powerful out of the box, custom rules have to be written in JavaScript, and the project hasn't seen much recent maintenance. The chapter frames these semantic-grep tools as most useful for two things: writing detectors that Slither's own rule engine can't easily express, and doing a fast initial inventory of a new codebase (how many external functions, how many libraries, and so on) before a deeper manual review begins.

Read the original article: https://x.com/officer_secret/status/1938239591946846266?s=20

CHAPTER 09

Semantic Grep & Solidity: In-Depth

The follow-up to the semantic-grep chapter, this time centered on Semgrep's Solidity rule support and how it stacks up against Pessimistic's own Slitherin detector set. Working through a shared batch of DeFi-exploit-derived rules, the authors found real overlap was much smaller than expected — only two rules actually caught the same thing across both tools — which they take as a sign the two approaches are more complementary than redundant, and promise a larger, 10,000-line comparison as a future follow-up. Guidance is given for auditors who want to write their own Semgrep rules or Slitherin detectors, including a candid note that AI-assisted rule generation, especially for regex-heavy matching, is useful for speeding up drafting but still needs a human to verify the result rather than being trusted outright. The bottom line offered is that both tools are strong at filtering and customizing a search, but their underlying approach and rule syntax are different enough that knowing both is more valuable than picking a favorite.

Read the original article: https://x.com/officer_secret/status/1969316151462105515?s=20

CHAPTER 10

Read-only Reentrancy: In-Depth

A dedicated follow-up to the original reentrancy chapter, this one built specifically around the read-only variant: a view function that, because it doesn't modify state, is often left completely unguarded even though it can still be called mid-reentry and return numbers that no longer reflect reality. The chapter treats this as a systematization of knowledge rather than a first-principles introduction, leaning on the earlier reentrancy piece for the base mechanics and building on top of it with the specific conditions under which a protocol's own price or exchange-rate getter becomes exploitable by a downstream integrator who trusts it mid-callback. The practical takeaway is that the checks-effects-interactions pattern, useful as it is, isn't automatically sufficient once other contracts are allowed to read a protocol's state during an external call — guarding view functions used as oracles by other contracts deserves the same scrutiny as guarding the state-changing functions themselves.

Read the original article: https://x.com/officer_secret/status/1939027625470566888?s=20

CHAPTER 11

AMM (Automatic Market Makers) Integration Tips

Focused less on AMM math in the abstract and more on the specific integration traps that show up when a project builds on top of Uniswap V2 or V3. The headline tip is to always verify that a call into *uniswapV3SwapCallback* actually originates from the genuine pool contract, either by calling the factory or by recomputing the pool's address deterministically via `CREATE2`, since skipping that check lets an attacker spoof being a pool and walk off with funds. The chapter also flags Slither's guidance on safe min/max square-root price ranges as a reference worth keeping on hand when reviewing swap logic. As with the rest of the series, it's framed as accumulated field experience from years of auditing AMM integrations rather than a tutorial on how constant-product pricing works, and points readers toward the companion protocol-specific chapters on Convex, Curve, Balancer, and Compound for deeper coverage of individual DeFi building blocks.

Read the original article: https://x.com/officer_secret/status/1939421043287789915?s=20

CHAPTER 12

Short Types in Solidity: Rare Tricks Uncovered

Looks at the trade-offs of using sub-256-bit integer types (like uint128 or uint96) instead of the default uint256. Packed correctly into a single storage slot alongside other small fields, short types can meaningfully cut gas costs on both storage writes and reads, which is why they show up constantly in gas-optimized DeFi code. The chapter's real focus is on the auditor's side of that trade-off: short types introduce their own overflow and truncation surface that uint256 arithmetic doesn't have to worry about in the same way, and casting between sizes carelessly is an easy way to silently lose precision or wrap a value. It treats short-type usage as a signal worth double-checking rather than a red flag on its own — the gas savings are real, but every explicit or implicit downcast in the vicinity deserves a second look during review.

Read the original article: https://x.com/officer_secret/status/1940081800493125990?s=20

CHAPTER 13

Gas Gauge: Pressure Control

Introduces Gas Gauge, a research tool purpose-built to detect out-of-gas denial-of-service vulnerabilities in Ethereum smart contracts — the class of bug where a function's execution can be blocked indefinitely because it isn't designed to tolerate running out of gas partway through. The chapter walks through the tool's three main components as described in its release paper and reports the authors' own hands-on impressions after using it, framing Gas Gauge as a solid complement to Slither rather than a replacement for it. It suggests running the two together as a 'dual check' pre-audit step: Slither's broader detector coverage catches the usual suspects, while Gas Gauge is aimed squarely at the gas-exhaustion pattern that general-purpose static analyzers don't always model well.

Read the original article: https://x.com/officer_secret/status/1940783903926526096?s=20

CHAPTER 14

Arbitrary Calls Auditing Tips

Centers on the newly released Arbitrary Call detector added to Slitherin, Pessimistic's own extended set of Slither detectors, and what it's built to catch: functions that let a caller pass in an arbitrary target address and have the contract execute a low-level call to it, most dangerously when that target can be an attacker-supplied token contract used to call transfer, transferFrom, or approve against a victim's own tokens. The Slitherin detectors are described as an automated version of the checks already baked into Pessimistic's internal audit checklist, meant to flag likely issues for a human reviewer rather than to make a final call on their own. The chapter also notes the tool's integration into Audit Wizard, a browser-based auditing IDE that filters out dependency noise from raw Slither output and bundles Slitherin's extra detectors alongside it for broader coverage.

Read the original article: https://x.com/officer_secret/status/1941149544625877150?s=20

CHAPTER 15

EVM Limitations & Assembly Auditing Tips

Closes out a three-part series covering gas optimization, a reentrancy-focused checklist, and EVM-level constraints in turn. This final part is aimed at the parts of an audit that touch inline assembly and the lower-level behavior of the EVM itself, arguing that a solid audit has to go beyond checking business logic and also verify that the contract's economic and game-theoretic incentives actually hold up — that token-issuance or staking mechanics behave the way the project's documentation claims they do. It ties back into Pessimistic's own Slitherin detector project as one of the concrete outputs of turning these lower-level review habits into automatable checks, and encourages readers to treat inline assembly blocks as an automatic escalation point during review, since Solidity's usual safety guarantees don't apply once code drops into raw Yul or EVM opcodes.

Read the original article: https://x.com/officer_secret/status/1941888977264398474?s=20

CHAPTER 16

Math, Solidity & Gas Optimizations

Opens the same three-part series with a focus on the intersection of arithmetic and gas cost. It walks through the standard toolbox of Solidity gas-optimization techniques — choosing appropriately sized data types, packing related storage variables into shared slots, minimizing on-chain computation and unnecessary array use, and reserving inline assembly for cases where the savings clearly justify the reduced readability — while framing every one of these choices as a security decision as much as a cost one. The chapter's underlying argument is that gas optimizations and correctness bugs live in the same code paths more often than developers expect: a change made purely to save gas, like packing two fields into one slot or moving a loop's storage reads into a local variable, can just as easily introduce a subtle arithmetic or ordering bug if it isn't reviewed with the same rigor as the original logic.

Read the original article: https://x.com/officer_secret/status/1942619535036596669?s=20

CHAPTER 17

ERC20 Integration Tips

Catalogs the ways real-world ERC20 tokens quietly diverge from the textbook standard and break integrations that assume every token behaves identically. It covers fee-on-transfer tokens, where the amount received is less than the amount sent because the token itself deducts a cut; tokens that skip returning a boolean from transfer entirely, which trips up code that checks the return value without accounting for tokens that never return one; and elastic-supply or rebasing tokens, where a holder's balance can change outside of any transfer they initiated, meaning totalSupply bookkeeping has to be handled with care. It also flags a subtler compliance gap some tokens have around resetting an allowance to zero before setting it to a new non-zero value, a minor standards violation that can still break integrations that assume strict ERC20 conformance. As with the rest of the integration-tips series, the guidance is framed as a systematization of knowledge drawn from years of auditing real token integrations rather than a first introduction to the ERC20 standard itself.

Read the original article: https://x.com/officer_secret/status/1943365175148494994?s=20

CHAPTER 18

Solidity Checklist & Reentrancy Attack

The middle part of the three-part gas/security/assembly series, pairing a condensed general-purpose audit checklist with a revisit of reentrancy in that broader context. Rather than reintroducing the reentrancy mechanics covered earlier in the book, this chapter treats it as one item on a longer list an auditor should be running through on every review — alongside the access-control, arithmetic, and external-call checks that come up across the whole series — and frames the point of a checklist as consistency: making sure a reviewer under time pressure still covers the same ground on every engagement instead of relying purely on intuition. Audits are framed here as serving multiple purposes at once: catching security risks before deployment, satisfying regulatory and client-trust expectations, and verifying that a contract's economic design actually produces the incentives its documentation promises.

Read the original article: https://x.com/officer_secret/status/1944082862669451338?s=20

CHAPTER 19

Initializing, Proxy, Oracles & Multi-Chain

Covers a grab-bag of auditor's notes that tend to surface together on upgradeable, oracle-dependent, multi-chain projects. On initialization, the recurring risk is a proxy contract's initializer being callable more than once, or callable by anyone before the intended deployer gets to it, since Solidity constructors don't run in the proxy's own storage context. On oracles, it stresses checking price decimals and 'dimensionality' consistently across every arithmetic operation, and warns against pricing an asset purely off the current token balance held by a contract at a single point in time. The multi-chain section is built around a concrete lesson from the Moonwell exploit: after the Nomad bridge was compromised, its wrapped bridge-tokens were deprecated, but Moonwell's Chainlink price feed for the underlying asset never reflected that the bridged token itself was now worthless, letting borrowers extract loans against effectively valueless collateral — a reminder that a price feed can be technically accurate for the wrong asset.

Read the original article: https://x.com/officer_secret/status/1944440837720433037?s=20

CHAPTER 20

Tokens, EIP-712 & Meta-Transactions

Explains EIP-712, the standard for structuring, hashing, and signing typed data so wallets can show users a readable summary of what they're actually signing instead of an opaque hex blob, and how that standard underpins the meta-transaction pattern. In a meta-transaction, a user (the signer) signs off on an intended action without needing any ETH of their own, while a separate relayer submits the actual transaction and covers the gas, with a proxy or relay contract validating the signature before forwarding the call — letting gasless or sponsored transactions work without breaking the signer's own security guarantees. The chapter connects this to the wider permit-style approval pattern (EIP-2612) that lets ERC20 approvals happen via a signed message instead of a separate on-chain transaction, and flags the auditor's job as verifying that signature domains, nonces, and replay protection are all wired correctly, since a meta-transaction system is only as safe as its signature-validation logic.

Read the original article: https://x.com/officer_secret/status/1945075193791062310?s=20

CHAPTER 21

AAVE V3 DeFi Integration Tips

Distills years of auditing Aave-based integrations into practical guidance for projects building on top of the protocol. Aave is described as a non-custodial liquidity market where suppliers earn interest and borrowers post over-collateralized positions, with the protocol's over-collateralization requirement being the core mechanism that makes trustless, permissionless lending viable in the first place. The chapter frames itself as a systematization of knowledge rather than an Aave tutorial, aimed at teams who are integrating Aave's lending markets into their own product and need to know which assumptions are safe to make about collateral factors, liquidation thresholds, and variable-versus-stable interest rates when building on top of the protocol rather than auditing Aave's own core contracts.

Read the original article: https://x.com/officer_secret/status/1945988581425582421?s=20

CHAPTER 22

AAVE V3 DeFi Integration: Specifications

A companion specifications document to the main AAVE V3 chapter, condensing the accumulated integration advice into a more checklist-like reference. It's built for readers who already understand Aave's over-collateralized lending model and want the concrete integration rules on hand while reviewing or building an Aave-connected contract, rather than the fuller explanatory framing of the tips chapter. As with the Chainlink VRF specifications companion earlier in the book, the two AAVE chapters are meant to be read together: the tips chapter for context on why each rule matters, and the specifications chapter as the quick-reference version for use during an actual review.

Read the original article: https://x.com/officer_secret/status/1969654966420742300?s=20

CHAPTER 23

Compound v2 DeFi Integration

Covers Compound v2 integration in depth, including a discussion of how Compound III (Comet) differs from earlier iterations. A key practical note is that Compound's functions historically returned an error code rather than reverting outright, so integrators had to explicitly check for a non-zero error rather than relying on a transaction-level revert — a pattern that changed in later versions, making it important to know which behavior a given deployment actually implements. It covers the mechanics of cTokens: they can be transferred and used as collateral elsewhere as ordinary ERC20-compatible tokens, but only so long as the transfer wouldn't push the account's overall liquidity below zero, and their underlying-token value grows block by block as interest accrues. Reward accounting via COMP token distribution is flagged as another area integrators need to model correctly rather than assume is automatic. Compound v2 gets this dedicated deep dive specifically because it remains the version most integrations actually build against, based on its share of total value locked relative to newer iterations.

Read the original article: https://x.com/officer_secret/status/1946735750461976926?s=20

CHAPTER 24

Compound v2 DeFi Integration: Specifications

The condensed specifications companion to the main Compound v2 chapter, restating the concrete integration checks — error-code versus revert-based failure handling, the collateral-liquidity requirement on cToken transfers, approval requirements before supplying tokens, and the fact that underlying-token value per cToken increases roughly per block — in a tighter reference format for use during an actual review. It notes that the deeper Compound III specific checks were planned as a further follow-up given how much the newer version's architecture diverges from v2, and frames this specifications chapter, like its AAVE and Chainlink VRF counterparts, as the quick-lookup half of a matched explanatory-plus-reference pair.

Read the original article: https://x.com/officer_secret/status/1970540452668449028?s=20

CHAPTER 25

Convex Finance DeFi Integration Tips

Covers integration guidance for Convex Finance, a yield-boosting layer built on top of Curve that lets CRV holders and Curve liquidity providers earn boosted staking rewards without having to lock their own tokens directly into Curve's veCRV system. The chapter treats Convex as effectively a second layer of DeFi composability sitting on top of Curve, meaning integrators need to reason about both protocols' assumptions at once, and it's framed as accumulated field experience from auditing projects that build against Convex's staking and reward contracts rather than a general explainer of what Convex is. As with the other DeFi-integration chapters in the book, it points readers who need the underlying pool mechanics toward the companion CurveV1 chapter for the pieces Convex itself builds on top of.

Read the original article: https://x.com/officer_secret/status/1947076917695578411?s=20

CHAPTER 26

CurveV1 Integration Tips

Focuses on the practical challenge of querying Curve pools correctly given how many pool types and registry contracts the protocol has accumulated over time. The chapter recommends relying on Curve's MetaRegistry rather than the older, more limited Registry contract, since MetaRegistry aggregates every pool type — stable-swap, crypto-swap, lending, meta-pools, factory-created pools, and pools Curve itself deployed — behind a single consistent set of getters (things like `get_base_pool`, `get_underlying_coins`, and fee lookups), which spares an integrator from having to special-case each registry's quirks individually. Each child registry has its own handler contract implementing the shared MetaRegistry interface, which is what lets an integration written once continue to work as Curve adds new pool types over time. Getting the wrong pool, coin ordering, or fee assumption from an outdated registry call is flagged as one of the more common integration mistakes worth specifically guarding against in review.

Read the original article: https://x.com/officer_secret/status/1947420089076355428?s=20

CHAPTER 27

Auditing Tips for NFT Projects

Surveys the security landscape specific to NFT projects, drawing on the history of NFT hacks and exploitation to make the case that NFT-specific risks deserve their own review lens rather than being treated as a subset of general ERC20-style token auditing. It stresses that NFT users and integrators alike need to understand these risks to protect their assets, and situates the chapter within the wider Convex/Curve/Balancer integration-tips series as the entry covering non-fungible rather than fungible assets. The underlying theme, consistent with the book's earlier randomness chapter, is that NFT minting logic which relies on weak or predictable on-chain randomness for trait or rarity assignment remains one of the most common and most avoidable classes of NFT-specific vulnerability.

Read the original article: https://x.com/officer_secret/status/1947913929172066575?s=20

CHAPTER 28

BalancerV1 Integration Tips

Distills close to two years of experience auditing Balancer V1 integrations. Balancer is presented as a protocol that lets users swap tokens instantly and earn fees by providing liquidity across multiple weighted pools rather than the fixed 50/50 pools of simpler AMMs, and the chapter's main practical tip is to route through batch swaps wherever possible: they're generally cheaper than chaining separate single swaps together, and they widen the set of token pairs an integration can actually reach, since a direct pool for two specific tokens doesn't always exist. As with Balancer's peer chapters on Curve and Convex, the guidance is framed as accumulated audit experience aimed at teams integrating Balancer's pools into their own contracts rather than a general introduction to how weighted-pool AMMs work.

Read the original article: https://x.com/officer_secret/status/1948367210121986053?s=20

CHAPTER 29

Arbitrum: Basic Features, Technical Details and Differences from Ethereum

Introduces Arbitrum as a Layer 2 network built by Offchain Labs on top of Ethereum mainnet, offering much faster and cheaper transactions while remaining EVM-compatible enough that most Ethereum DeFi contracts can be deployed with little to no modification. Transactions are forwarded to the Arbitrum Sequencer, which reports results roughly a second after submission rather than waiting for Ethereum's own block time, and average transaction costs are described as a small fraction of typical L1 gas fees. The chapter walks through a framework for understanding how L2s like Arbitrum differ structurally from L1, aimed at giving auditors who are used to reviewing Ethereum mainnet contracts the mental model they need before reviewing an Arbitrum-native or Arbitrum-deployed project, since some assumptions that hold on L1 — particularly around finality timing and sequencer trust — don't carry over unchanged.

Read the original article: https://x.com/officer_secret/status/1949923526347002261?s=20

CHAPTER 30

Rebase Tokens, Rounding Errors & DoS Attacks: Unraveling the Mystery

Examines the family of bugs that show up around rebasing tokens, whose balances are periodically recalculated rather than only changing on explicit transfers, which makes their accounting meaningfully harder to reason about than a standard ERC20. Three linked risks are called out: the sheer conceptual complexity confuses users about what they actually hold, the balance-adjustment math introduces rounding errors that can quietly accumulate into real discrepancies over time, and that same complexity opens the door to denial-of-service bugs where an attacker disrupts normal contract operation by exploiting an edge case in the rebase logic. The KyberSwap hack, which resulted in roughly \$53 million in losses, is cited as a real-world case where a rounding error in the contract's arithmetic played a role. On root causes, the chapter traces the problem back to Solidity's lack of any floating-point type: integer division always truncates rather than rounding, so $5 / 2$ silently becomes 2 instead of 2.5, and every calculation involving fractional values has to be designed around that constraint deliberately. Suggested mitigations include delaying division until the very end of a calculation to preserve precision, distributing token amounts sequentially so no 'dust' is left unaccounted for, and using tools like MythX to catch rounding issues before they reach production.

Read the original article: https://x.com/officer_secret/status/1950262581131674008?s=20

CHAPTER 31

Leap Seconds & Time in Solidity

A short, deliberately offbeat chapter on why time is trickier in Solidity than it looks. It notes that the EVM counts time in seconds and that developers generally have two choices for time-based logic: keying off the current block number or off the current block timestamp, each with its own trade-offs around precision and manipulability by miners or validators. The chapter's real hook is the leap-second problem: because the Earth's rotation is gradually and unpredictably slowing, the difference between atomic time and observed solar time (UT1-UTC) can only be measured after the fact, never predicted in advance, and small changes in angular velocity compound over time into a growing, irregular correction that has to be periodically patched into civil timekeeping. It's offered less as a practical audit checklist item and more as a reminder that even something as seemingly fixed as 'a second' rests on assumptions that occasionally get revised, which is exactly the kind of edge case that time-dependent contract logic can't always afford to ignore.

Read the original article: https://x.com/officer_secret/status/1950961563491340404?s=20

CHAPTER 32

Ethereum Alarm Clock Exploit: Final Thoughts

Walks through the hack of the Ethereum Alarm Clock, a service that let users schedule a transaction for future execution by pre-paying for its gas, with independent 'TimeNodes' claiming and executing scheduled transactions in exchange for a bounty. The exploit itself worked by repeatedly calling the cancel function on scheduled transactions with an inflated fee attached; because the protocol refunded gas fees on cancellation, a bug in that refund logic let attackers extract more value back out than they had actually spent, effectively minting profit out of the cancellation flow. Before getting into the exploit mechanics, the chapter takes a detour into why services like this attracted privacy-conscious users in the first place, using it as a jumping-off point to discuss representative-sampling deanonymization — the way a real-world transaction (say, meeting someone in person to trade crypto) combined with an on-chain record of activity at that same moment can be enough to link a wallet address back to a real identity, a reminder that OpSec discipline matters well beyond the smart-contract layer itself.

Read the original article: https://x.com/officer_secret/status/1952803773753012656?s=20

CHAPTER 33

25 Tips for Solidity Code Auditors

A running, crowdsourced list of practical workflow tips gathered from the wider auditor community rather than a single narrative essay. It covers editor tooling — running VS Code on an iPad with a Blink Shell setup, Solidity-aware extensions, a Sourcegraph-connected AI project analyzer, and note-taking categorization conventions like @audit/@audit-info/@audit-issue comment tags — alongside process advice such as forcing yourself to explain every function's purpose in plain language before judging its correctness, and studying past audit findings and mind-mapping techniques to build pattern recognition faster. It also rounds up a long list of niche but genuinely useful tools: pre-built security properties for commonly forked DeFi protocols, MEV/sandwich-attack research compilations, Pessimistic's own Slither detectors alongside SolCurity, fuzzers like Pyrometer and Sporalyzer, and browser-level protection via the Web3Antivirus extension. The list is explicitly framed as a living document that grows as the community contributes to it, rather than a fixed and finished set of 25 tips.

Read the original article: https://x.com/officer_secret/status/1956894536573022652?s=20

CHAPTER 34

Rust & Blockchain Security

Looks at Rust's growing role across blockchain tooling and node software, and what that means for security review. Rust's compile-time memory-safety guarantees remove a whole category of bugs that plague C and C++ codebases, but the chapter is careful to note that safety guarantee only holds inside 'safe' Rust — any block marked unsafe opts back out of those guarantees and deserves the same scrutiny a reviewer would give raw C, which is exactly why tools like Cargo Geiger (covered earlier in the book's NEAR chapter) exist to flag how much unsafe code a crate and its dependency tree actually contain. It also touches on dependency-chain risk: a Rust project's overall security posture depends heavily on the crates it pulls in, making tools like Cargo Audit for known-vulnerable dependencies and Clippy for catching common mistakes part of a baseline review workflow rather than optional extras, particularly for the growing set of blockchain clients, indexers, and off-chain infrastructure being built in Rust alongside on-chain Solidity code.

Read the original article: https://x.com/officer_secret/status/1957530563277123700?s=20

CHAPTER 35

Variant Analysis & Glider Tool

Introduces variant analysis as a research technique and Glider as a tool built specifically to run it at scale across the smart-contract ecosystem. The core idea behind variant analysis is simple but powerful: once a known bug or vulnerability pattern has been found in one contract, the same underlying code shape is very likely to have been copied, forked, or independently reinvented elsewhere, so the natural next question is how to search every other deployed contract for that same shape. Glider is presented as a code-query engine that treats every abstraction — a line of code, a variable, a function — as a queryable object, letting a researcher run variant and taint analysis across all verified open-source contracts on integrated EVM chains at high speed. Deliberately using ordinary Python syntax for its query language rather than inventing a new one is called out as a specific design choice meant to flatten the learning curve and make the tool more accessible to auditors who already know Python, while still leaving room to integrate external tools into the same query pipeline.

Read the original article: https://x.com/officer_secret/status/1958159196769120715?s=20

CHAPTER 36

Optimizing the Auditing Process in Your Auditing Company

Shifts focus from individual audit technique to running an audit firm efficiently at the organizational level. Standardization is presented as the foundational step: building comprehensive checklists for every phase of an engagement — planning, fieldwork, reporting, and final review — each covering risk assessment, evidence gathering, and compliance verification, so that audit quality doesn't depend entirely on which individual auditor happens to be assigned. A central knowledge hub, built with something as simple as SharePoint, GitBook, Docusaurus, or Google Sites, is recommended as a place to store findings, historical audit data, and reusable templates so auditors aren't reinventing the same checks project after project; the chapter estimates this kind of centralization alone can cut time spent on repetitive tasks by up to 30%. On scheduling, it recommends building a master audit calendar for the year that prioritizes high-risk areas and builds in regulatory or client deadlines up front, rather than reacting to engagements one at a time as they arrive.

Read the original article: https://x.com/officer_secret/status/1971538884556194065?s=20

CHAPTER 37

How Does a Compiler Actually Work?

A primer on what actually happens between writing Solidity source code and getting deployable bytecode, aimed at auditors who want to understand the pipeline their code passes through rather than treating the compiler as a black box. It walks through the standard stages: lexical analysis, where raw source text is broken into a stream of tokens representing keywords, identifiers, and operators; syntactic analysis, where a parser checks that token stream against the language's grammar rules and builds an abstract syntax tree representing the code's logical structure; semantic analysis, where the compiler checks things like type correctness and proper control-flow usage against that tree; and finally code generation, where the tree is translated into lower-level instructions, historically human-readable assembly that an assembler then turns into real machine code before a linker stitches multiple compiled units into one final executable. The chapter frames this pipeline as directly relevant to security work: understanding how the Solidity compiler actually lowers high-level constructs into EVM bytecode makes it much easier to reason about compiler-introduced behavior, optimizer quirks, and the kinds of subtle discrepancies between source-level intent and on-chain execution that a purely surface-level code review can miss.

Read the original article: https://x.com/officer_secret/status/2074390205511946679?s=20

Colophon

Officer's Big Auditing Book was compiled as a single reference from a longer-running series of articles on Solidity and smart-contract security, spanning reentrancy, oracle and randomness security, protocol-specific DeFi integration notes, tooling (Slither, Echidna, Gas Gauge, Glider, Semgrep), gas and EVM-level review, and general auditing workflow. Each chapter in this edition is an independent, rewritten summary; the original article for every chapter is linked at the end of that chapter for readers who want the full original text, code samples, and resource lists.

Once again — thank you to the pessimistic.io auditors and to the hexens.io team.

If you'd like to support future work: 0x1191b7d163bde5f51d4d2c1ac969d514fb4f4c62 or officercia.eth